I2C Master/Slave IP Core

with APB Register Interface, 10-bit Addressing,
Clock Stretching and Multi-Master Arbitration

Design and Verification Report

B. Srishanth Reddy

Electronics and Communication Engineering

November 2026

Tools: Verilog-2005 RTL · Icarus Verilog 12 · GTKWave · Yosys 0.52 · GitHub Actions CI

Abstract

This report describes an I2C (Inter-Integrated Circuit) master and slave IP core that I designed in synthesizable Verilog and verified with self-checking testbenches. The master handles START, repeated START and STOP, 7- and 10-bit addressing, reads and writes with ACK/NACK, a programmable SCL prescaler checked against the Standard-mode and Fast-mode timing limits, an SCL-low timeout and a bus-recovery command, and it detects lost arbitration when two masters talk at once. The slave has a programmable address, a 16 x 8 register file and clock stretching. An AMBA APB wrapper lets a CPU drive the master through six registers; commands written while the master is busy, or invalid commands, are rejected and flagged. Six testbenches with a real open-drain bus model and a protocol/timing monitor run 28 tests (including 450 constrained-random transactions with a scoreboard) and 3134 checks, and all of them pass. The regression also runs in GitHub Actions on every push. A deliberately injected bug is caught by the testbench. Waveforms were checked in GTKWave, and the screenshots in this report are real captures of the laptop screen.

1. Introduction

I2C is a two-wire serial bus (SCL clock, SDA data) used to connect sensors, EEPROMs, RTCs and PMICs to a processor. Both lines are open-drain: a device can only pull a line low, and a pull-up resistor makes it high. This wired-AND behaviour is what allows several devices, and even several masters, to share the bus. After my Configurable UART IP Core, I wanted to build a protocol that has addressing, acknowledgements and multi-master behaviour, so I chose I2C.

Objectives:

- Synthesizable master and slave in Verilog, connected through a real open-drain bus model
- Master: START / repeated START / STOP, 7- and 10-bit addressing, read/write, ACK/NACK, Standard/Fast-mode prescaler, arbitration-loss detection, SCL timeout
- Slave: programmable address, register file, clock stretching
- APB register interface with command-ready/reject handling and bus recovery, so the core can be used in an SoC
- I2C timing compliance (tLOW, tHIGH, tSU;STA, tHD;STA, tSU;DAT, tHD;DAT, tSU;STO, tBUF) checked in simulation
- Self-checking directed, random and error-case testbenches with protocol assertions, CI, and waveforms in GTKWave

2. I2C protocol basics

Condition	Definition
START (S)	SDA goes high to low while SCL is high
STOP (P)	SDA goes low to high while SCL is high
Repeated START (Sr)	A START without a STOP before it; used to change direction (write pointer, then read)
Data bit	SDA changes only while SCL is low and is sampled while SCL is high, MSB first
ACK / NACK	9th bit: the receiver pulls SDA low (ACK) or leaves it high (NACK)

7-bit address	First byte = A6..A0 + R/W
10-bit address	First byte = 11110 A9 A8 R/W, second byte = A7..A0
Clock stretching	A slave holds SCL low to make the master wait
Arbitration	A master that sends 1 but reads 0 on SDA has lost and must release the bus

3. Architecture

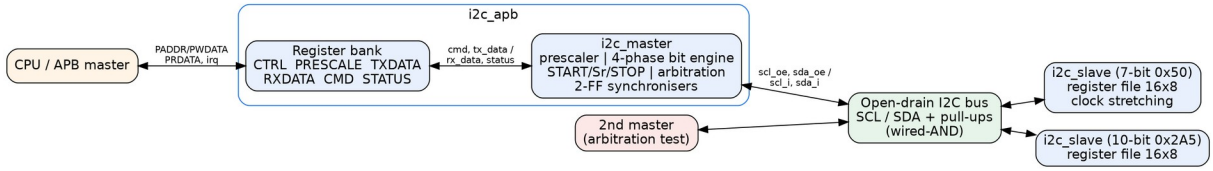


Figure 1: Block diagram of the IP core and the verification setup

The design has three modules. i2c_master is the bus engine, i2c_slave is a register-file slave, and i2c_apb wraps the master with an APB register bank. Every SCL/SDA driver is open-drain: the output enable pulls the line low and otherwise leaves it at 'z'. In the testbenches, the bus wires are declared tri1, which models the pull-up resistors.

3.1 Bit timing

Each bit, START and STOP is split into four phases of PRESCALE+1 clock cycles. In phase 1 the counter waits until the synchronised SCL is high, which adds 3 clocks (2-flop synchroniser plus one compare) to the high time. The measured period is therefore f_SCL = f_clk / (4 x (PRESCALE + 1) + 3). At 50 MHz, PRESCALE = 124 gives 99.40 kHz and PRESCALE = 32 gives 370.37 kHz. To meet the Standard-mode tSU;STA, tHD;STA and tSU;STO limits, the START phases 1-2 and the STOP phases 1-3 last 2 x (PRESCALE + 1) clocks.

Phase	SCL	SDA / action
ph0	driven low	master puts the next bit on SDA (or releases it to read)
ph1	released	the counter is held at 0 until SCL is really high; this supports clock stretching and clock synchronisation between masters. SDA is sampled and arbitration checked at the end of ph1
ph2	high	START: SDA pulled low here; STOP: SDA released here
ph3	driven low	end of bit

3.2 Master FSM

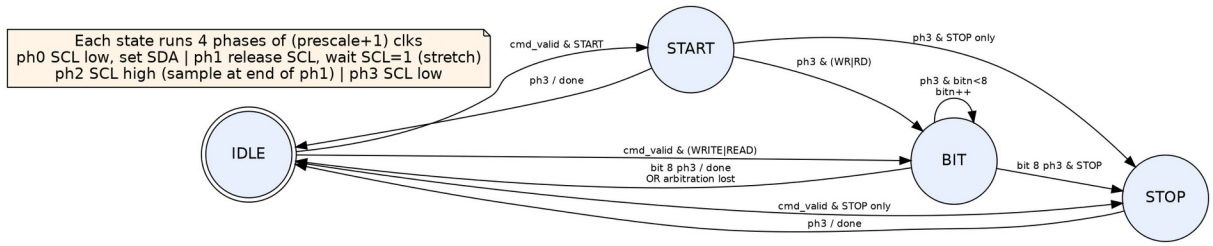


Figure 2: Master state machine

A command can contain START, one WRITE or READ byte and STOP, so a whole byte transfer needs one command. During a WRITE, if the master releases SDA (sends 1) but the synchronised SDA is 0 at the sampling point, another master is driving the bus. The master then sets arb_lost, releases both lines in the same cycle and goes back to IDLE. A START/STOP monitor drives bus_busy, so software can wait for a free bus before retrying.

3.3 Slave FSM

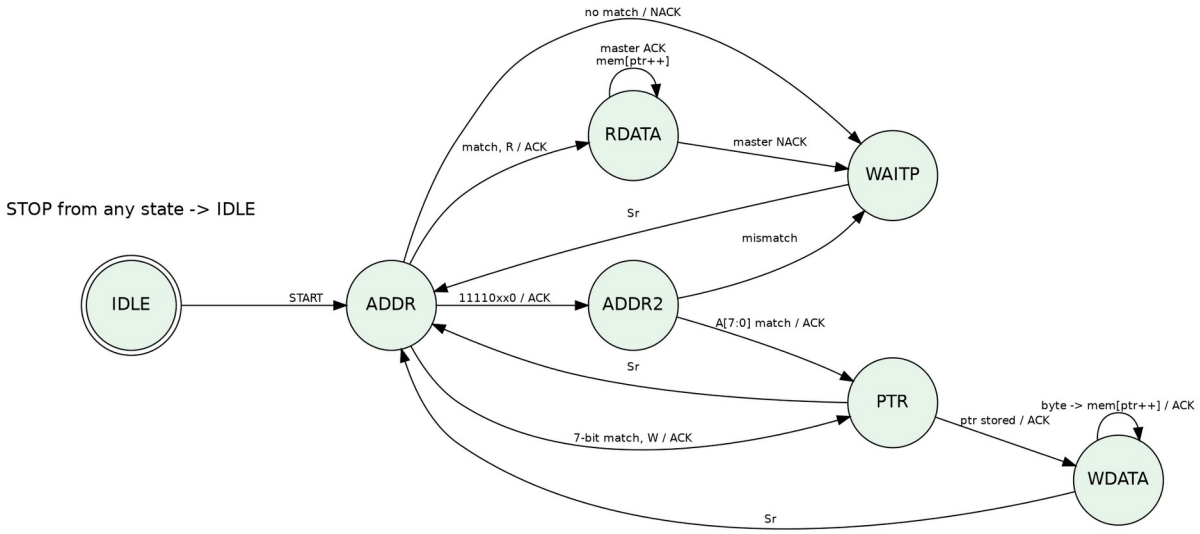


Figure 3: Slave state machine

The slave detects START and STOP from synchronised SCL/SDA. It shifts data in on SCL rising edges and changes SDA only after SCL falling edges. The first byte after the address is the register pointer, and it increments after every data byte. For 10-bit mode, the slave answers the header 11110 A9 A8 0, checks the low address byte, and remembers the match, so a later Sr with 11110 A9 A8 1 starts a read. When clock stretching is enabled, the slave pulls SCL low for stretch_cycles clocks after each ACK bit.

3.4 APB register map

Offset	Name	Access	Description
0x00	CTRL	RW	[0] irq_en
0x04	PRESCALE	RW	[15:0] SCL prescaler, reset 124
0x08	TXDATA	RW	[7:0] byte to send
0x0C	RXDATA	RO	[7:0] last byte read
0x10	CMD	WO	[0] START [1] STOP [2] READ [3] WRITE [4] NACK [5] RECOVER; accepted only when cmd_ready = 1 and the command is valid
0x14	STATUS	RO/W1C	[0] busy [1] rx_nack [2] arb_lost [3] done (W1C) [4] bus_busy [5] cmd_err (sticky, W1C) [6] cmd_ready [7] timeout [8] scl [9] sda; reset 0x340

3.5 Command handling and bus recovery

- cmd_ready = master idle and no command pending. A CMD write while cmd_ready = 0 is dropped and sets cmd_err; nothing appears on the bus. Software must poll STATUS.cmd_ready (or STATUS.done) before writing CMD.
- Invalid commands (READ and WRITE together, or RECOVER with any other bit) are also rejected with cmd_err. irq = irq_en & (done | cmd_err).
- RECOVER sends 9 SCL pulses with SDA released and then a STOP, which frees a slave that is holding SDA low (for example after the master was reset in the middle of a read).
- If SCL stays low for more than TIMEOUT_CLKS clocks (default 1 250 000 = 25 ms at 50 MHz) the master aborts, releases both lines and sets STATUS.timeout.

4. Timing compliance

tb_i2c_timing runs transfers at several PRESCALE values. A passive bus monitor (tb/i2c_monitor.v) measures every timing parameter on the bus and compares the worst case with the UM10204 limits. Measured worst-case values (laptop log logs/tb_i2c_timing.log):

Parameter	Std-mode P=124	Limit	Fast-mode P=32	Limit	Result
tLOW	5000 ns	>= 4700	1320 ns	>= 1300	PASS
tHIGH	5060 ns	>= 4000	1380 ns	>= 600	PASS
tSU;STA	5060 ns	>= 4700	1380 ns	>= 600	PASS
tHD;STA	5000 ns	>= 4000	1320 ns	>= 600	PASS
tSU;DAT	2500 ns	>= 250	660 ns	>= 100	PASS
tHD;DAT	60 ns	>= 0	60 ns	>= 0	PASS
tVD;DAT	2560 ns	<= 3450	720 ns	<= 900	PASS
tSU;STO	5060 ns	>= 4000	1380 ns	>= 600	PASS
tBUF	17560 ns	>= 4700	4680 ns	>= 1300	PASS

The earlier Fast-mode setting PRESCALE = 30 (393.70 kHz) gives tLOW = 1240 ns, which is below the 1300 ns Fast-mode limit. The timing testbench checks that the monitor flags exactly this one violation. PRESCALE = 32 (370.37 kHz) is therefore the fastest compliant setting at 50 MHz. Also, in the first version START/STOP setup and hold were about 2.5 µs, below the Standard-mode limits; doubling the START/STOP phases fixed this. Rise and fall times are not modelled.

5. Verification

Six self-checking testbenches compare register contents, received bytes, status flags and measured timing against expected values, and print PASS or FAIL. The protocol monitor is connected to the bus in every testbench and acts as a set of assertions written in plain Verilog so that they work in Icarus: SDA must not change while SCL is high inside a byte, SCL/SDA must never be X or Z, and the timing limits above. tb_i2c_robust checks command rejection, invalid commands, reset during a transfer with stuck-SDA recovery, the SCL timeout and a monitor self-test. tb_i2c_rand runs 150 seeded constrained-random transactions per seed (random target including an empty address, direction, pointer, length, data, speed and stretching) against a scoreboard model of both register files. To make sure the testbench can really detect

errors, a negative test builds the slave with +define+INJECT_BUG, which flips bit 0 of every byte written to the register file; the core testbench must then fail, and it does. scripts/run_all.ps1 (Windows) and scripts/run_all.sh (Linux) run everything, and a GitHub Actions workflow runs run_all.sh on every push and fails the build on any error.

Testbench	Test	Result
tb_i2c_core	1. 7-bit write, 4 bytes, 100 kHz	PASS
tb_i2c_core	2. 7-bit read, repeated START, ACK/NACK	PASS
tb_i2c_core	3. NACK: no slave at address 0x58	PASS
tb_i2c_core	4. 10-bit write to 0x2A5	PASS
tb_i2c_core	5. 10-bit read with repeated START	PASS
tb_i2c_core	6. 10-bit NACK: wrong low address byte	PASS
tb_i2c_core	7. clock stretching by slave (600 clk = 12 us)	PASS
tb_i2c_core	8. Fast-mode 370 kHz (prescale=32)	PASS
tb_i2c_arb	1. arbitration lost detected, winner completes	PASS
tb_i2c_arb	2. retry after arbitration loss	PASS
tb_i2c_arb	3. arbitration in data phase	PASS
tb_i2c_apb	1. APB register access	PASS
tb_i2c_apb	2. APB write transfer	PASS
tb_i2c_apb	3. APB read transfer	PASS
tb_i2c_apb	4. NACK status	PASS
tb_i2c_timing	1. Standard-mode 100 kHz	PASS
tb_i2c_timing	2. Fast-mode, fastest compliant	PASS
tb_i2c_timing	3. prescale 30 violates Fast-mode	PASS
tb_i2c_timing	4. formula check only (no limits)	PASS
tb_i2c_robust	1. command while busy is rejected (cmd_err)	PASS
tb_i2c_robust	2. invalid commands rejected, bus untouched	PASS
tb_i2c_robust	3. software flow with cmd_ready polling	PASS
tb_i2c_robust	4. reset during read -> SDA stuck low -> RECOVER	PASS
tb_i2c_robust	5. SCL held low by slave -> timeout -> RECOVER	PASS
tb_i2c_robust	6. protocol assertion self-test (illegal SDA edge)	PASS
tb_i2c_rand_seed1	1. 150 random transactions, seed=1584433774	PASS
tb_i2c_rand_seed2	1. 150 random transactions, seed=-392144044	PASS
tb_i2c_rand_seed3	1. 150 random transactions, seed=1463049771	PASS
negative	INJECT_BUG: corrupted register write	Bug caught

Summary lines from logs/summary.log (28 tests, 3134 checks, 0 errors):

```
tb_i2c_core: SUMMARY tb_i2c_core: 8/8 tests PASS, 56 checks, 0 errors
tb_i2c_core: RESULT: ALL TESTS PASSED
tb_i2c_arb: SUMMARY tb_i2c_arb: 3/3 tests PASS, 18 checks, 0 errors
tb_i2c_arb: RESULT: ALL TESTS PASSED
```

```
tb_i2c_apb: SUMMARY tb_i2c_apb: 4/4 tests PASS, 20 checks, 0 errors
tb_i2c_apb: RESULT: ALL TESTS PASSED
tb_i2c_timing: SUMMARY tb_i2c_timing: 4/4 tests PASS, 20 checks, 0 errors
tb_i2c_timing: RESULT: ALL TESTS PASSED
tb_i2c_robust: SUMMARY tb_i2c_robust: 6/6 tests PASS, 24 checks, 0 errors
tb_i2c_robust: RESULT: ALL TESTS PASSED
tb_i2c_rand_seed1: SUMMARY tb_i2c_rand: 1/1 tests PASS, 1009 checks, 0 errors
tb_i2c_rand_seed1: RESULT: ALL TESTS PASSED
tb_i2c_rand_seed2: SUMMARY tb_i2c_rand: 1/1 tests PASS, 984 checks, 0 errors
tb_i2c_rand_seed2: RESULT: ALL TESTS PASSED
tb_i2c_rand_seed3: SUMMARY tb_i2c_rand: 1/1 tests PASS, 1003 checks, 0 errors
tb_i2c_rand_seed3: RESULT: ALL TESTS PASSED
NEGATIVE TEST: injected bug detected by testbench ... PASS
```

Measured values from the logs:

- tb_i2c_core: SCL period min 10060.0 ns max 10120.0 ns -> 99.40 kHz
- tb_i2c_core: longest SCL low time 12060.0 ns (normal low = 5000 ns)
- tb_i2c_core: SCL period min 2700.0 ns max 2760.0 ns -> 370.37 kHz
- tb_i2c_core: protocol monitor: 14 START, 4 Sr, 10 STOP, 428 bits, 0 assertion failures
- tb_i2c_arb: master A done at 10330 ns, arb_lost=1
- tb_i2c_arb: master B done at 26810 ns, arb_lost=0, rx_nack=0
- tb_i2c_arb: protocol monitor: 3 START, 0 Sr, 3 STOP, 93 bits, 0 assertion failures
- tb_i2c_robust: after reset: SCL=1 SDA=0 slave addressed=1
- tb_i2c_robust: after RECOVER: SCL=1 SDA=1 bus_busy=0 slave addressed=0
- tb_i2c_robust: STATUS = 2d8: timeout=1 busy=0 at 773050 ns
- tb_i2c_robust: protocol monitor: 10 START, 2 Sr, 5 STOP, 202 bits, 0 assertion failures
- tb_i2c_rand_seed1: 63 writes, 75 reads, 12 NACK probes, 463 data bytes, 50 to 10-bit slave, 31 with stretching
- tb_i2c_rand_seed1: protocol monitor: 225 START, 75 Sr, 150 STOP, 8109 bits, 0 assertion failures

6. Waveforms (GTKWave screenshots)

The figures below are real screenshots of the laptop screen. GTKWave opened the VCD files from the simulation runs, and scripts/gtkwave_shots.ps1 captured each view. The view scripts in scripts/gtkw/ set the signals, zoom and marker.

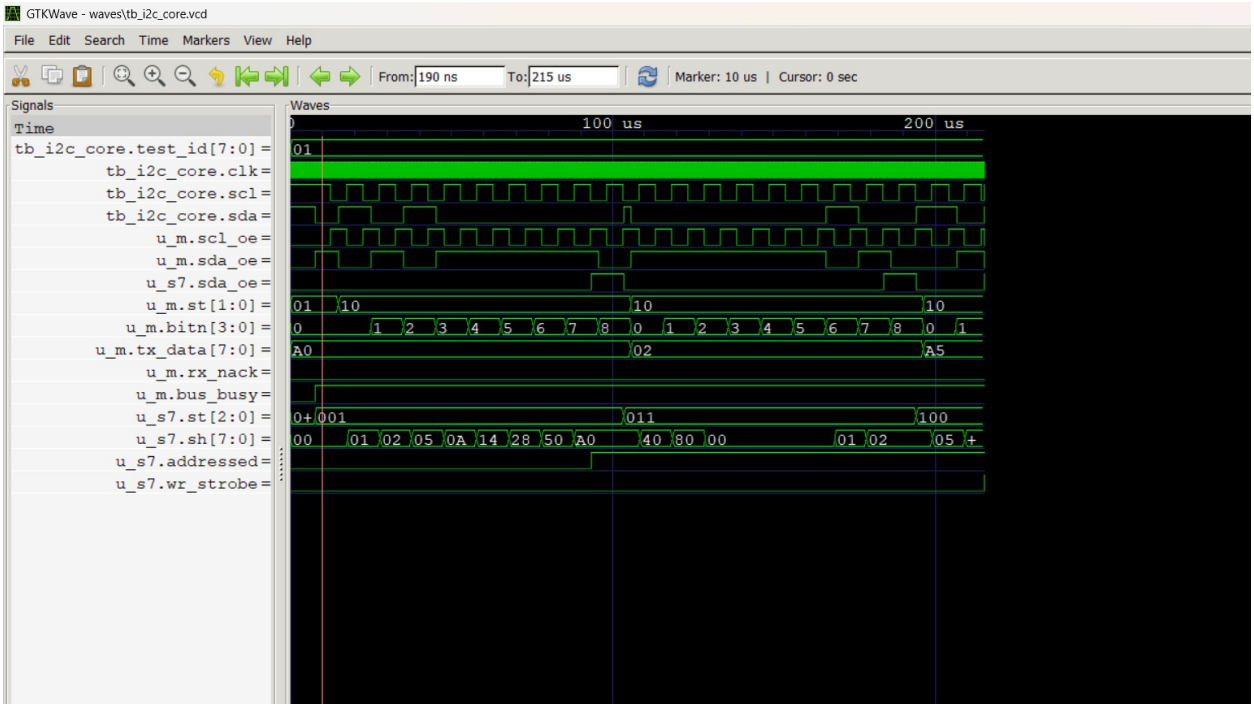


Figure 4: 7-bit write at 100 kHz: START, address 0xA0 + ACK, pointer, data bytes (tb_i2c_core test 1)

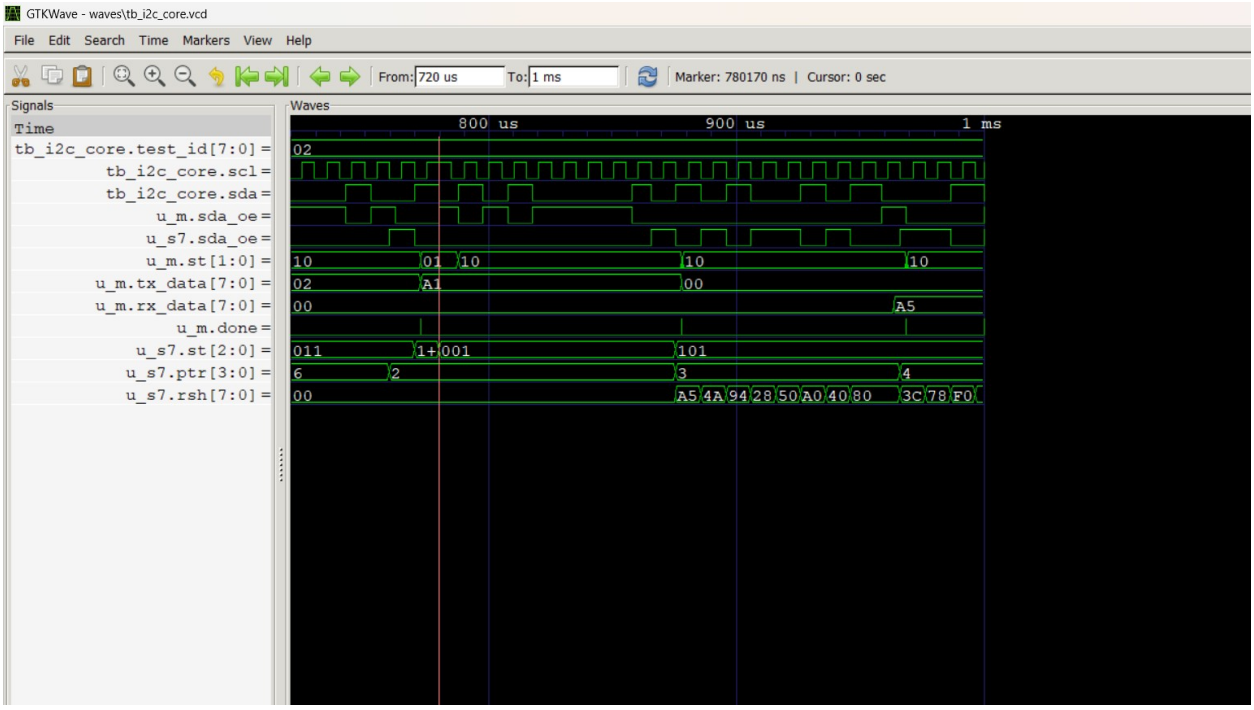


Figure 5: Read with repeated START: the slave drives data, the master ACKs and NACKs the last byte (test 2)

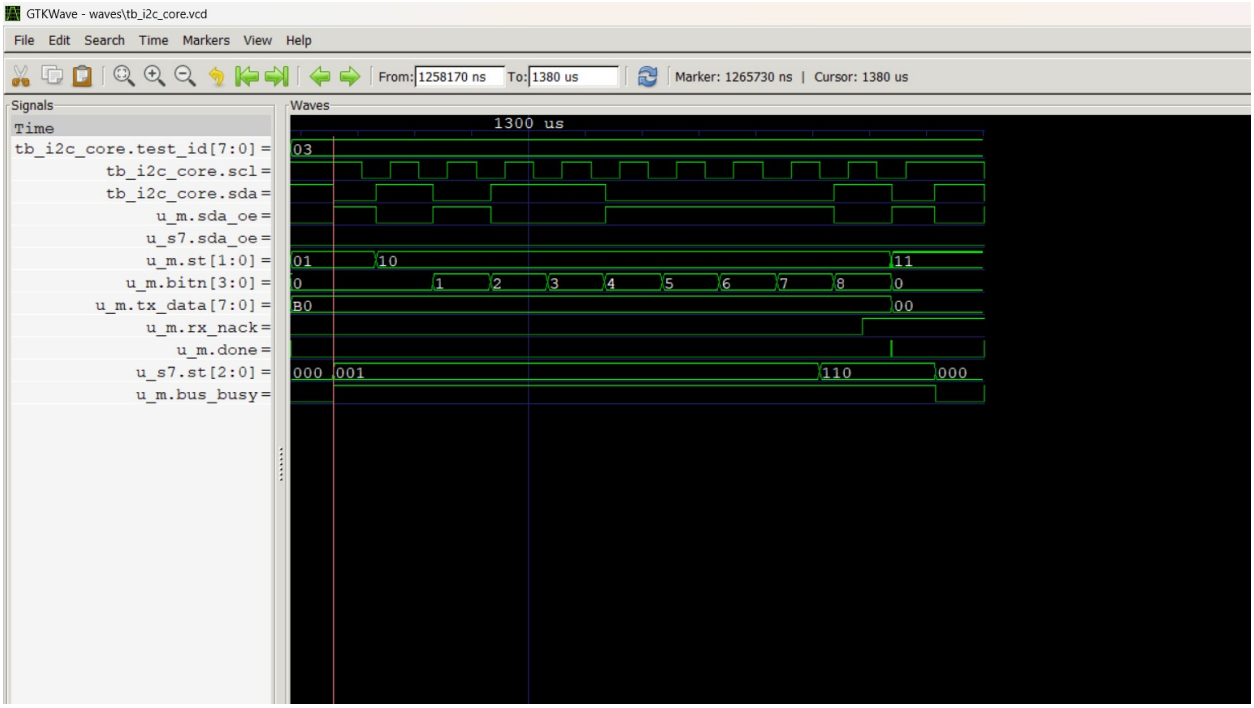


Figure 6: Address NACK: nobody answers 0x58, so SDA stays high on the 9th bit and rx_nack = 1 (test 3)

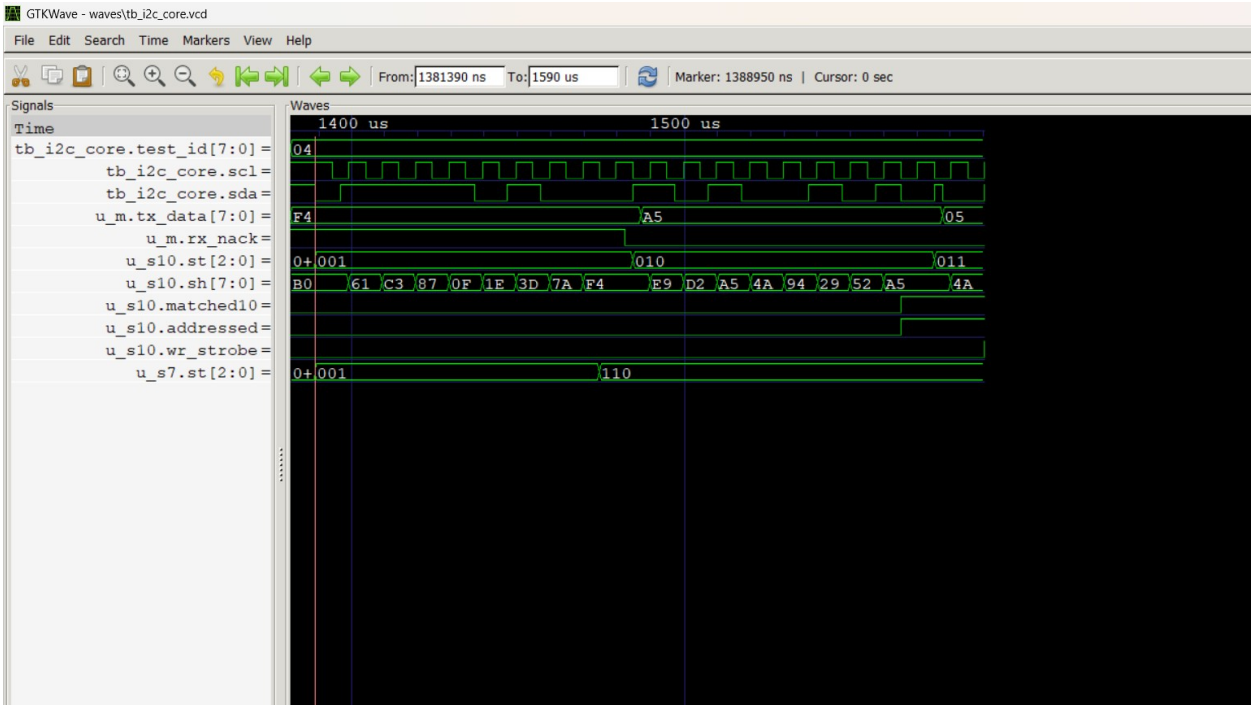


Figure 7: 10-bit addressing: header 0xF4 then low byte 0xA5; the 10-bit slave sets matched10 (test 4)

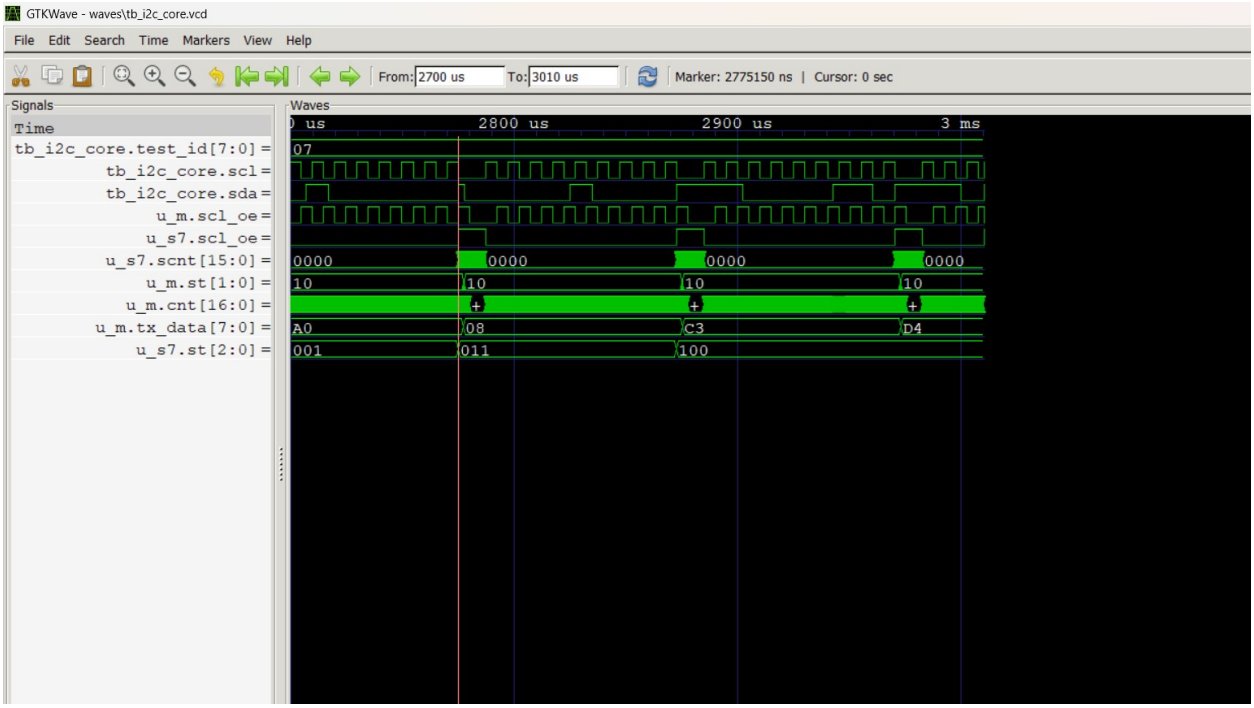


Figure 8: Clock stretching: the slave holds SCL low (scl_oe, scnt) and the master waits in phase 1 (test 7)

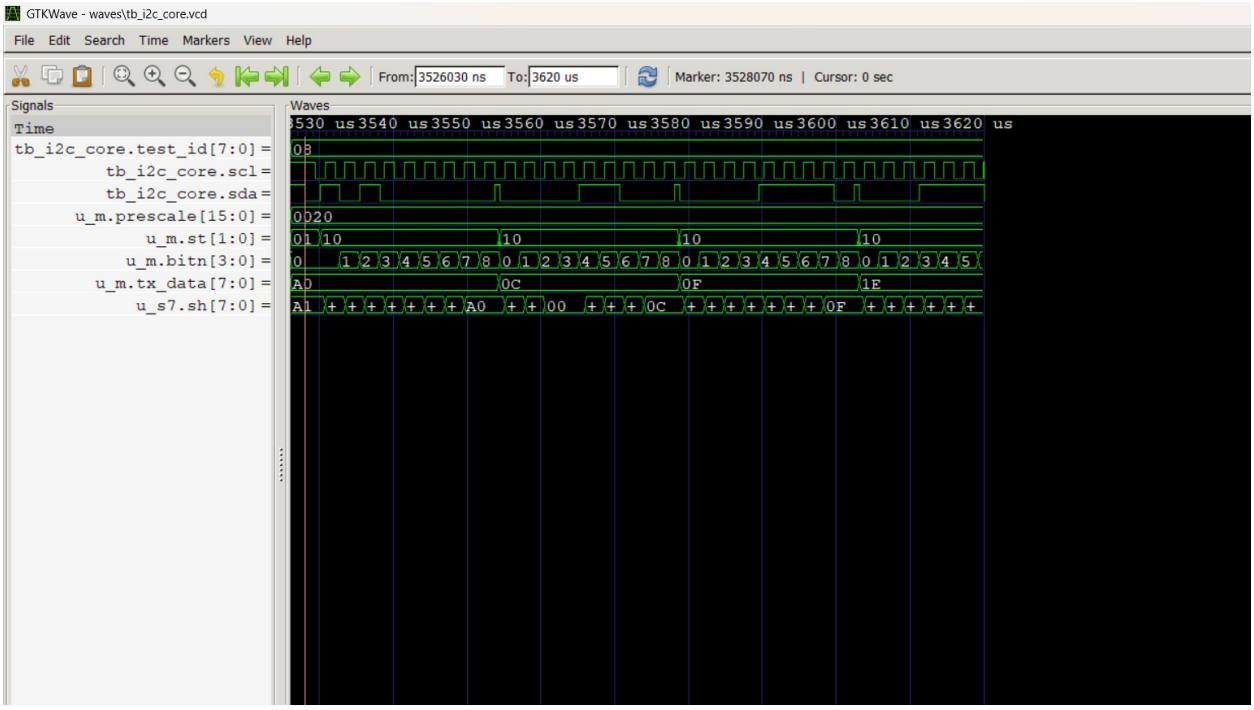


Figure 9: Fast-mode, 370 kHz with PRESCALE = 32 (test 8)

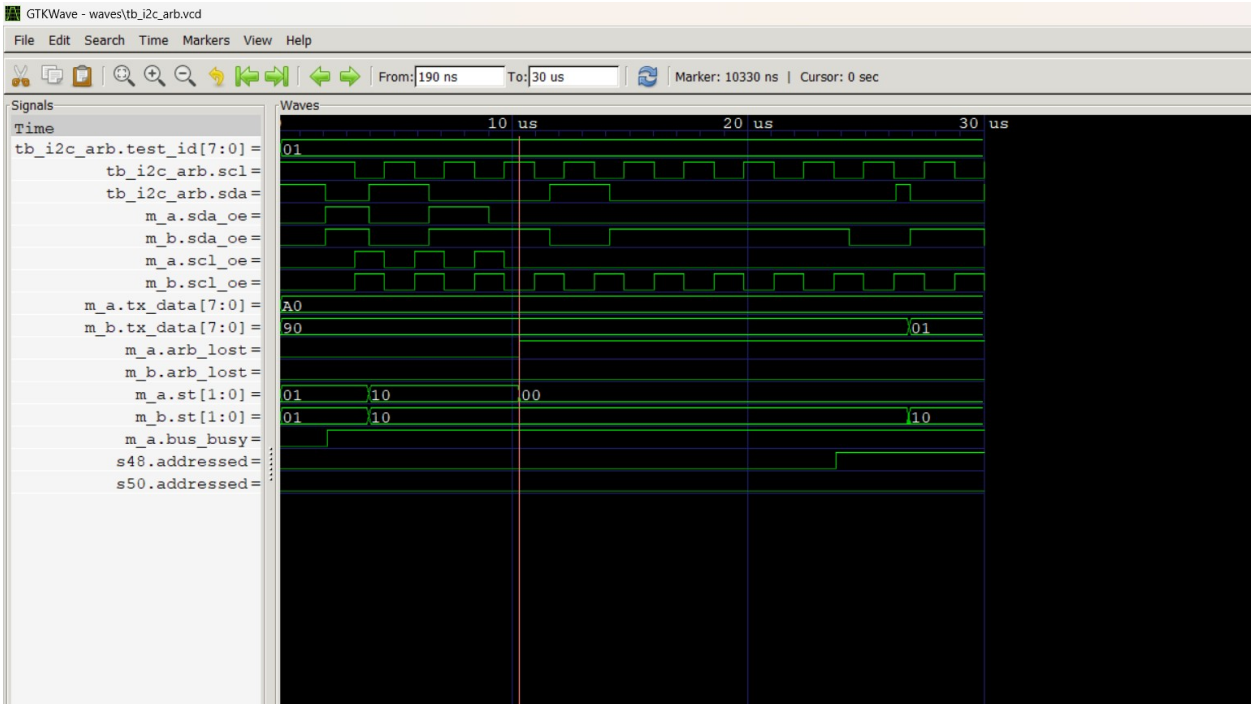


Figure 10: Multi-master arbitration: A sends 0xA0, B sends 0x90; A reads 0 while sending 1, sets arb_lost and releases the bus

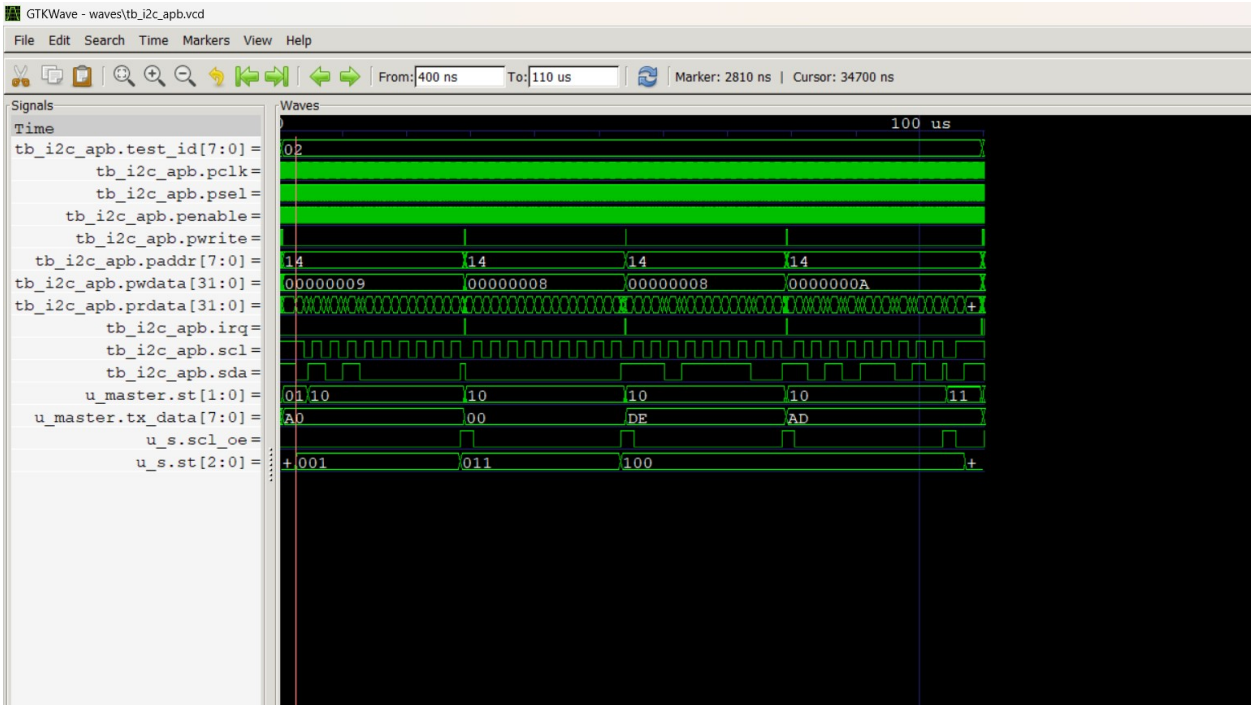


Figure 11: APB-driven transfer: APB writes to TXDATA/CMD, STATUS polling, irq, and the I2C bus with slave stretching

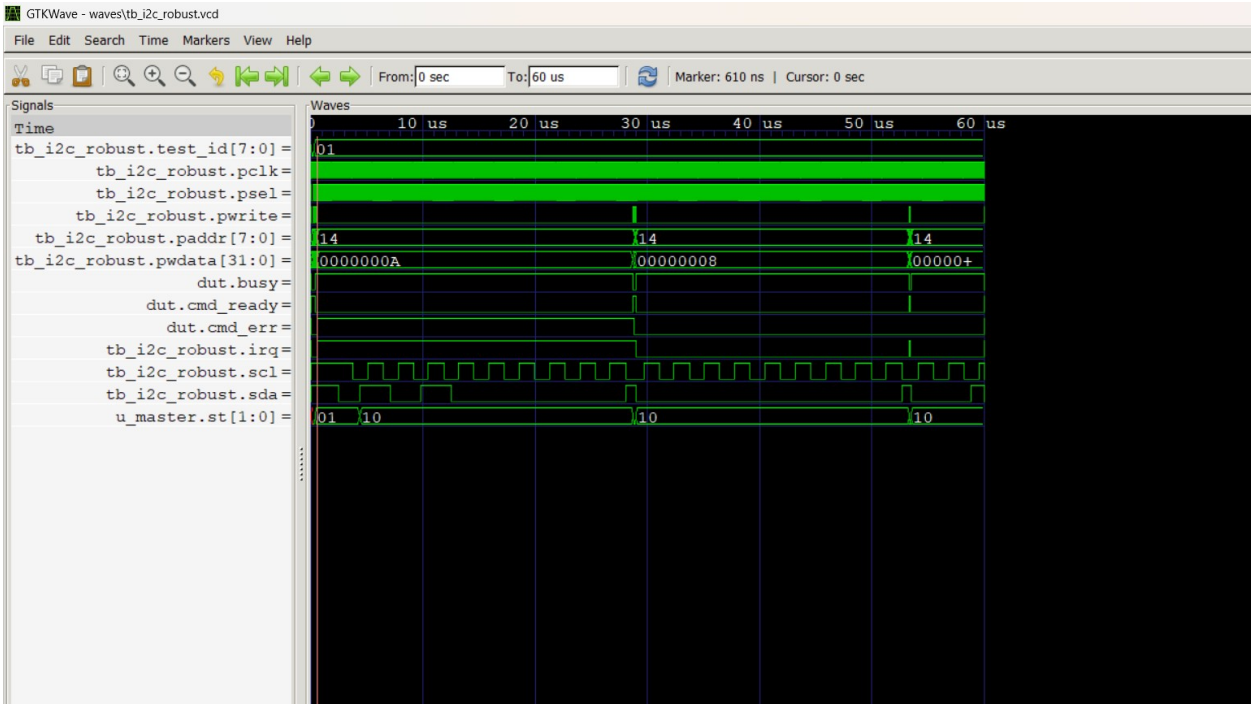


Figure 12: Command written while busy: cmd_err and irq go high, the running transfer continues, W1C clears cmd_err (tb_i2c_robust test 1)

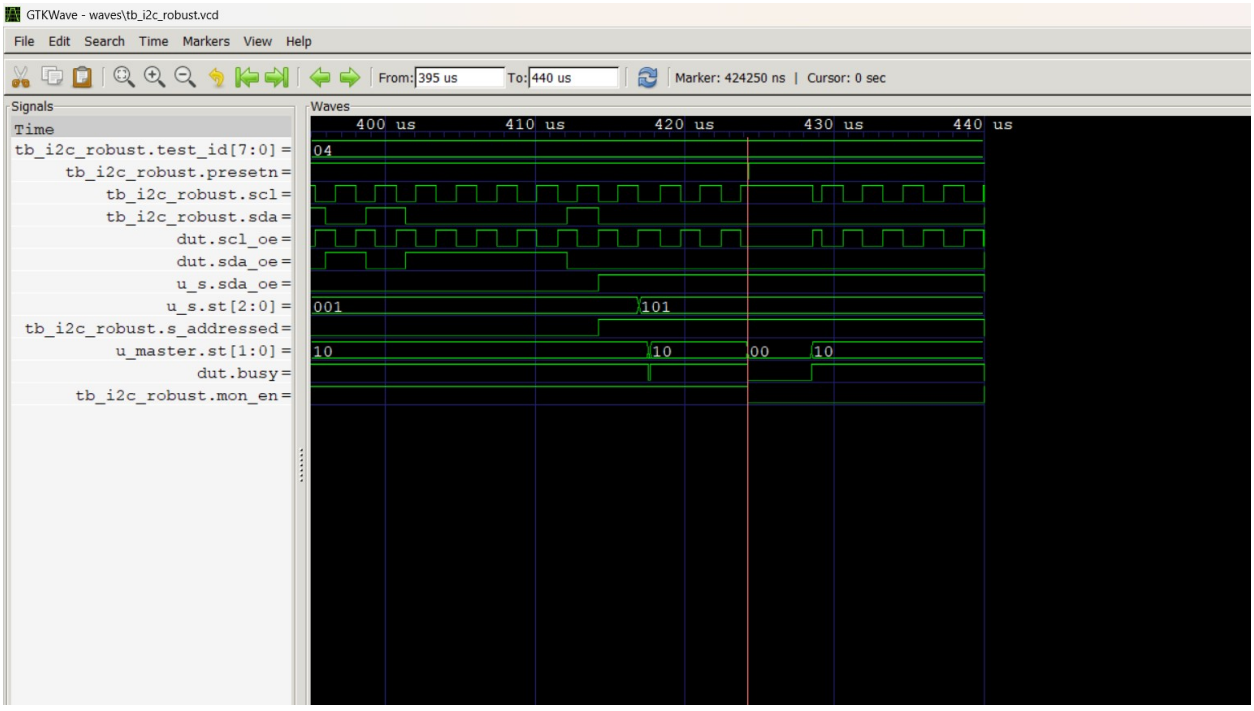


Figure 13: Master reset during a read: the slave keeps SDA low, so the bus is stuck (tb_i2c_robust test 4)

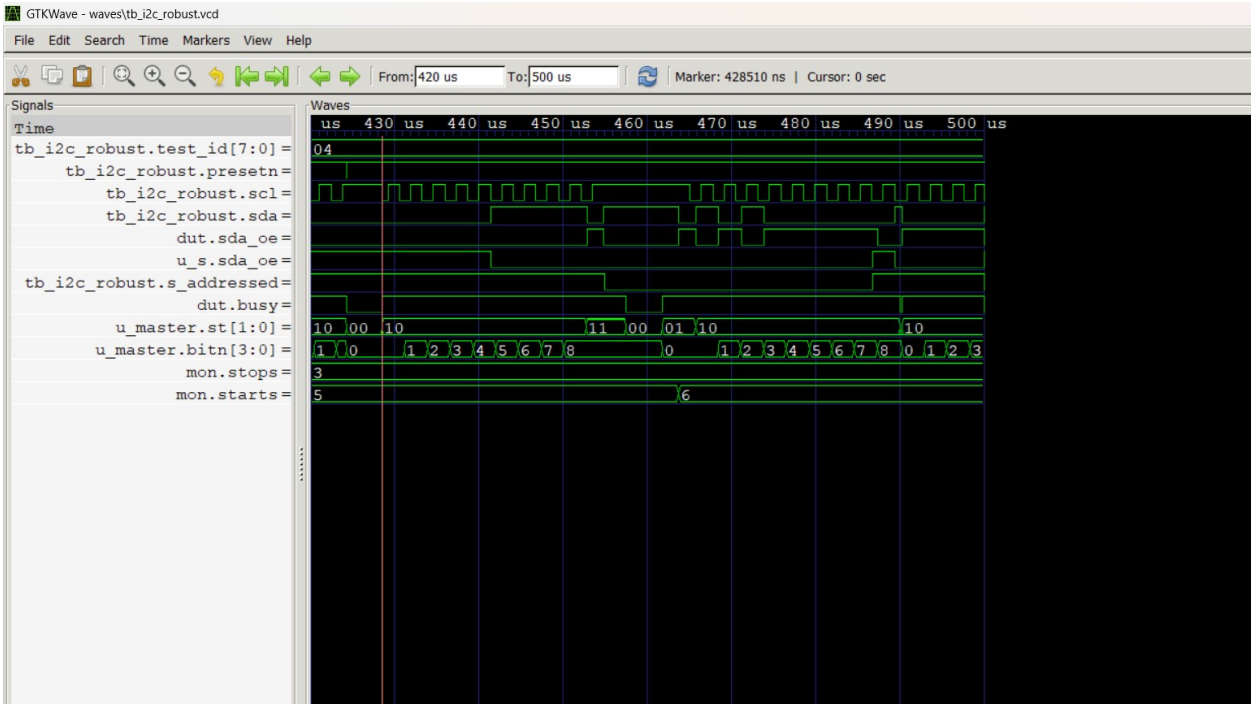


Figure 14: CMD.RECOVER: 9 SCL pulses with SDA released, STOP, then a normal transfer (tb_i2c_robust test 4)

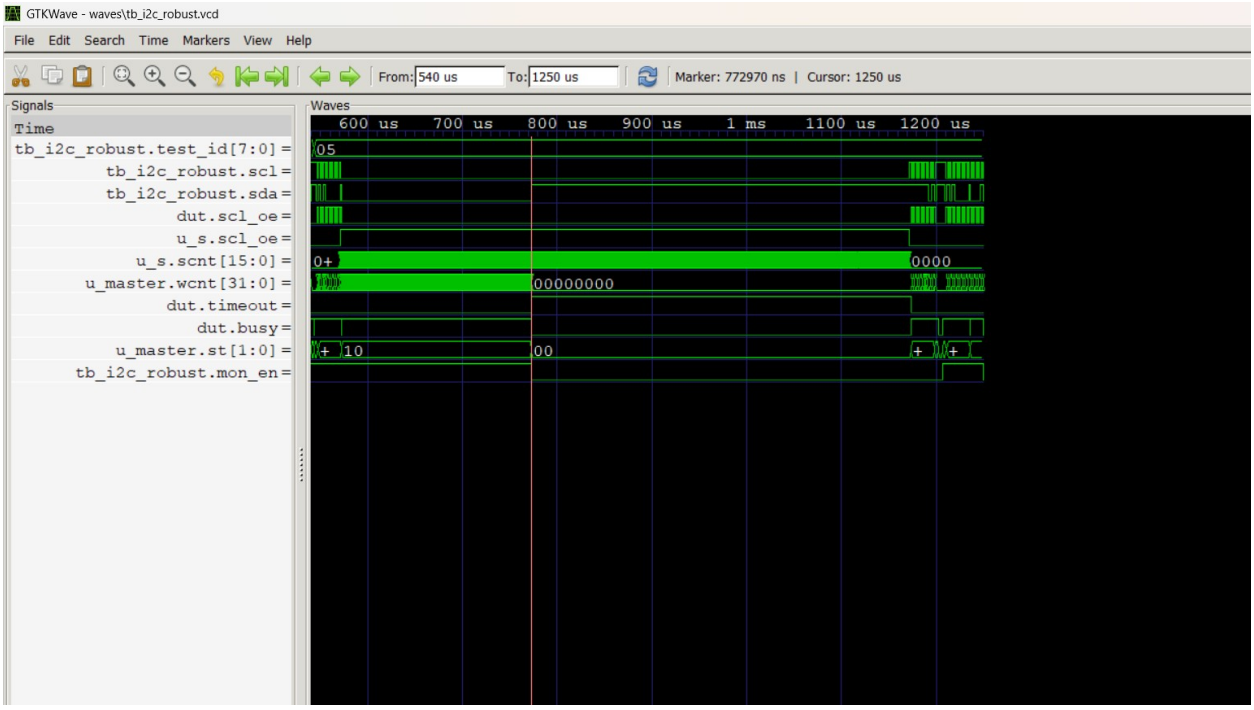


Figure 15: Slave holds SCL low for 600 µs: the master times out after 200 µs, then RECOVER (tb_i2c_robust test 5)

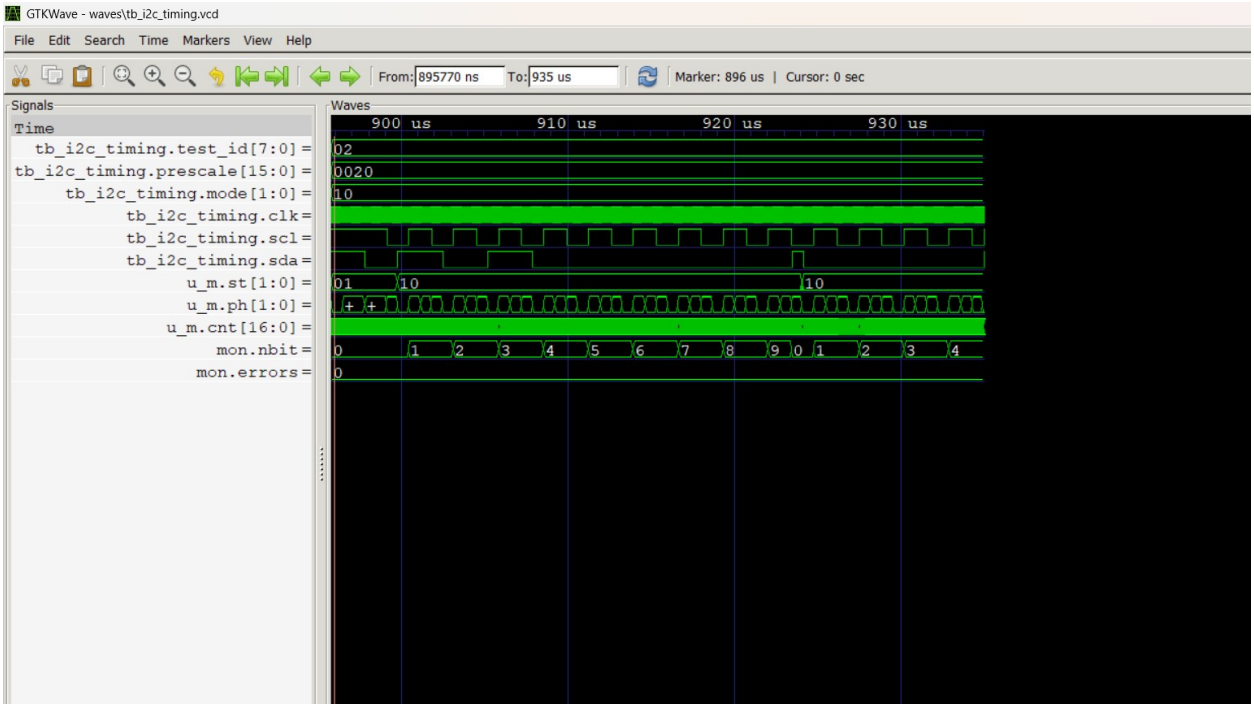


Figure 16: Fast-mode timing test, PRESCALE = 32: four phases per bit, monitor errors = 0 (tb_i2c_timing test 2)

7. Synthesis

Top	Generic cells	Flip-flops
i2c_master	399	57
i2c_slave (16 x 8 regs)	1293	186
i2c_apb (wrapper + master)	544	89

Yosys 0.52 'synth -flatten' with generic gates (syn/synth.ys). This shows the RTL is synthesizable; it is not a real cell library, so there are no area or timing figures. Yosys is not installed on the laptop, so this step ran separately on Linux, on the RTL before the timeout/recovery changes; it has not been re-run since.

8. Results and discussion

- All 28 tests in 6 testbenches pass with 0 errors over 3134 checks and 0 assertion failures, and the injected bug is detected. The same run passes in GitHub Actions.
- Measured SCL matches f_clk / (4(P+1)+3) exactly: 99.40 kHz at P = 124, 370.37 kHz at P = 32, 393.70 kHz at P = 30 and 1063.83 kHz at P = 10.
- Standard-mode at P = 124 and Fast-mode at P = 32 meet every timing limit checked; P = 30 violates Fast-mode tLOW, and the checker catches it.
- Commands written while busy and invalid commands are rejected and reported; a bus stuck by a reset in the middle of a read and an SCL held low too long are both recovered.
- With stretching on, the longest SCL low time was 12.06 µs against 5 µs normally, and every byte still transferred correctly. This shows the master waits for SCL.

- In arbitration, the losing master releases the bus in the same bit, and the winner finishes without any corruption.

9. Limitations and future work

- Byte-level master: software sends the address bytes; a transaction-level sequencer, FIFOs and DMA would reduce CPU load.
- No spike filter on SCL/SDA, and only one prescaler (no separate tLOW/tHIGH registers), so Fast-mode reaches 370 kHz rather than 400 kHz at 50 MHz. Rise/fall times are not modelled.
- Arbitration is not checked during the START condition itself.
- Simulation-only verification. Future work: FPGA validation on a board such as Basys 3 or Arty A7 with real I2C devices (EEPROM, sensor, RTC) and a logic analyser; a UVM environment with functional coverage; SVA properties proven with SymbiYosys; gate-level simulation.

10. Conclusion

I designed and verified a complete I2C master/slave IP core with an APB interface. It covers the key parts of the I2C specification: START/Sr/STOP, 7- and 10-bit addressing, ACK/NACK, Standard- and Fast-mode timing, clock stretching, multi-master arbitration, command rejection, timeout and bus recovery. Every result in this report comes from the simulation logs and real GTKWave screenshots.

References

- NXP Semiconductors, UM10204 I2C-bus specification and user manual, Rev. 7.0, 2021.
- ARM, AMBA 3 APB Protocol Specification v1.0, 2004.
- S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed., Pearson, 2003.